

```

# =====
# Rainfall Prediction System
# Vineesh Nandi
# =====
# HOW TO USE:
# 1. Go to https://colab.research.google.com
# 2. Create a New Notebook
# 3. Paste this entire code into a cell and run it (Shift+Enter)
# 4. It will download a real public weather dataset, train models,
#    and print out real accuracy/F1 metrics you can use on your resume.
# =====

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import accuracy_score, f1_score, classification_report, conf
from sklearn.impute import SimpleImputer
import warnings
warnings.filterwarnings('ignore')

# -----
# STEP 1: Load the dataset
# This is the "Rain in Australia" dataset - a well-known public
# weather dataset with ~145,000 rows of real weather station data.
# -----
url = "https://raw.githubusercontent.com/Sharvani-Marisetti/Rainfall-Prediction/m

try:
    df = pd.read_csv(url)
    print(f"Dataset loaded successfully: {df.shape[0]} rows, {df.shape[1]} column
except Exception as e:
    print("Primary URL failed, trying backup...")
    url2 = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/weatherAU
    df = pd.read_csv(url2)
    print(f"Dataset loaded successfully: {df.shape[0]} rows, {df.shape[1]} column

print("\nFirst few rows:")
print(df.head())
print("\nColumns:", list(df.columns))

# -----
# STEP 2: Data Cleaning & Preprocessing

```

```

# -----

# Drop rows where target variable is missing
df = df.dropna(subset=['RainTomorrow'])

# Select relevant numeric and categorical features
features = ['MinTemp', 'MaxTemp', 'Rainfall', 'Humidity9am', 'Humidity3pm',
            'Pressure9am', 'Pressure3pm', 'Temp9am', 'Temp3pm', 'WindSpeed9am',
            'WindSpeed3pm', 'Cloud9am', 'Cloud3pm', 'RainToday']

df_model = df[features + ['RainTomorrow']].copy()

# Encode categorical Yes/No columns
le = LabelEncoder()
df_model['RainToday'] = df_model['RainToday'].fillna('No')
df_model['RainToday'] = le.fit_transform(df_model['RainToday'])
df_model['RainTomorrow'] = le.fit_transform(df_model['RainTomorrow'])

# Handle missing values in numeric columns using median imputation
numeric_cols = [c for c in features if c != 'RainToday']
imputer = SimpleImputer(strategy='median')
df_model[numeric_cols] = imputer.fit_transform(df_model[numeric_cols])

print(f"\nAfter cleaning: {df_model.shape[0]} rows ready for modeling")
print(f"Class balance (RainTomorrow):\n{df_model['RainTomorrow'].value_counts(nor

# -----
# STEP 3: Train / Test Split
# -----

X = df_model[features]
y = df_model['RainTomorrow']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"\nTrain set: {X_train.shape[0]} samples")
print(f"Test set: {X_test.shape[0]} samples")

# -----
# STEP 4: Train Model (Random Forest)
# -----

```

```

model = RandomForestClassifier(
    n_estimators=200,
    max_depth=12,
    min_samples_split=5,
    class_weight='balanced', # handles class imbalance (more "No" than "Yes")
    random_state=42,
    n_jobs=-1
)

model.fit(X_train_scaled, y_train)

# -----
# STEP 5: Evaluate
# -----

y_pred = model.predict(X_test_scaled)

accuracy = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("\n" + "="*50)
print("RESULTS - copy these numbers into your resume")
print("="*50)
print(f"Accuracy: {accuracy*100:.2f}%")
print(f"F1-Score: {f1:.3f}")
print("\nFull classification report:")
print(classification_report(y_test, y_pred, target_names=['No Rain', 'Rain']))

# Feature importance - useful talking point in interviews
importances = pd.Series(model.feature_importances_, index=features).sort_values(ascending=False)
print("\nTop 5 most important features:")
print(importances.head())

# -----
# STEP 6: Save model (optional, for deployment later)
# -----

import joblib
joblib.dump(model, 'rainfall_model.pkl')
joblib.dump(scaler, 'scaler.pkl')
print("\nModel saved as rainfall_model.pkl")

```